

Treinamento de um Modelo de Linguagem Autorregressivo em Nível de Caractere com Textos de Machado de Assis

FABRICIO FERNANDES SANTANA
Departamento de Ciência da Computação
Universidade de Brasília
Matrícula: 262114527

Resumo—Este trabalho implementa, treina e compara duas configurações de modelos autorregressivos em nível de caractere, treinadas do zero com uma amostra de textos de Machado de Assis. Usando nanoGPT em versão fixada e o mesmo orçamento de atualizações em uma Tesla T4, investigamos como a redução conjunta de camadas, cabeças e dimensão das representações vetoriais se associa à previsão no teste. Em 6.400 janelas amostradas do fluxo de teste, o modelo de referência obteve perda de 1,2397 nats por token e perplexidade 3,455; a configuração reduzida obteve 1,3862 e 4,000. As métricas usam o vocabulário de caracteres, incluindo os tokens especiais de fim de documento e de caractere desconhecido. Incluindo representações posicionais, a configuração reduzida tem 69,8% menos parâmetros únicos, mas maior perda neste protocolo. O corpus é uma amostra de trabalho, sem alegação de representar a obra completa. Discute-se uma extensão por recuperação de evidências, sem atribuir capacidade conversacional aos modelos atuais.

Index Terms—modelos de linguagem, Transformer, nanoGPT, Machado de Assis, geração de texto, aprendizado profundo.

I. INTRODUÇÃO

Modelos de linguagem autorregressivos estimam a probabilidade do próximo token, a unidade de representação do texto, a partir de um prefixo. A arquitetura Transformer substitui recorrência por mecanismos de atenção, permitindo relacionar posições distintas da sequência [1]. Modelos da família GPT utilizam uma organização apenas com decodificador (*decoder-only*) e máscara causal: a previsão do próximo token depende somente dos tokens já disponíveis [2]. Esse princípio permite estudar modelagem de linguagem com arquiteturas menores que as dos modelos de grande escala.

O objetivo deste trabalho é construir um modelo de linguagem baseado na arquitetura GPT com conteúdo de Machado de Assis, utilizando o nanoGPT como código-base. O trabalho focaliza a implementação de um modelo de referência (*baseline*), a avaliação quantitativa e qualitativa de sua geração e a discussão de uma possível extensão para perguntas sobre o autor. A contribuição é um experimento didático, documentado e auditável em um corpus literário em português, não uma reprodução do GPT-2 original em escala ou tokenização.

O estudo responde à seguinte questão: sob o mesmo corpus, partições e orçamento de 5.000 atualizações, como a redução conjunta de profundidade, número de cabeças e dimensão das representações vetoriais se associa à perda amostral de teste e ao número de parâmetros? Para respondê-la, apresentamos

a preparação dos dados, duas configurações de modelo, a comparação quantitativa e a inspeção qualitativa de amostras.

II. FUNDAMENTAÇÃO

O Transformer utiliza atenção para combinar informações de diferentes posições [1]. Na variante do nanoGPT, cada token recebe uma representação vetorial aprendida (*embedding*), somada à representação de sua posição. A sequência passa por blocos de atenção causal e redes de propagação direta (*feed-forward*), com normalização e conexões residuais. Após a normalização final, uma projeção produz escores (*logits*) para o vocabulário; a função softmax os transforma nas probabilidades do próximo token [3].

Para uma cabeça de atenção, a operação é

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} + M \right) V. \quad (1)$$

As matrizes Q , K e V contêm, respectivamente, consultas, chaves e valores obtidos por projeções aprendidas das representações de entrada. O produto entre consultas e chaves determina os pesos usados para combinar valores; d_k é a dimensão das chaves. A máscara M vale zero nas posições permitidas e $-\infty$ nas futuras, anulando seus pesos após softmax. As cabeças calculam essas relações em paralelo, e suas saídas são concatenadas e projetadas [1]. O nanoGPT usa apenas blocos causais, sem o codificador e a atenção cruzada do Transformer original [3].

O treinamento ajusta os parâmetros θ minimizando a entropia cruzada. Para T tokens-alvo, com $x_{<t}$ representando o prefixo disponível na janela, a perda média e a perplexidade são

$$L = -\frac{1}{T} \sum_{t=1}^T \log p_\theta(x_t | x_{<t}), \quad \text{PPL} = \exp(L). \quad (2)$$

O logaritmo natural expressa a perda em nats por token; a perplexidade é adimensional. Neste trabalho, cada token representa um caractere ou um dos símbolos especiais de fim de documento (EOS) e de caractere desconhecido (UNK). Ambos participam da perda quando aparecem nos alvos. Essa convenção é mantida em todas as métricas reportadas.

A tokenização por caracteres segue o exemplo `shakespeare_char` do nanoGPT. O tutorial de Karpathy apresenta os componentes passo a passo, enquanto este experimento

adapta o código-base para o corpus e seu tokenizador [3], [4]. Uma janela de 256 tokens corresponde a aproximadamente 256 caracteres, descontados os tokens especiais. A tokenização por subpalavras, como BPE, pode representar unidades textuais maiores; suas perplexidades não são diretamente comparáveis às deste estudo, pois muda a unidade de predição [2].

Brown et al. investigam, no GPT-3, aprendizagem em contexto: exemplos e instruções no prefixo orientam a resposta sem atualizar os pesos na aplicação da tarefa [5]. Isso difere do treinamento por gradientes realizado aqui. A escala e o protocolo daquele estudo também diferem dos presentes; prever continuções de texto não demonstra, por si só, aprendizagem em contexto ou capacidade de responder perguntas.

III. DADOS E METODOLOGIA

A. Corpus e partições

O corpus deriva do pacote `machado` do NLTK, cujos metadados apontam para a coleção digital do MEC, produzida com participação do NUPILL/UFSC [6], [7]. Dos 246 arquivos de texto da fonte, quatro traduções foram excluídas; o mestre de trabalho reúne 242 itens: 112 indexados tratados como autorais e 130 contos avulsos. O inventário inclui contos, crítica, crônicas, romances, teatro e poesia.

Na seleção dos contos avulsos, mediu-se a fração de seus n-gramas distintos de 32 caracteres encontrada em cada uma de sete coletâneas, tomando-se a maior cobertura. A comparação usou o corpo após a nota “Publicado originalmente”, normalizado em uma sequência alfanumérica sem diacríticos. Coberturas a partir de 0,85 foram classificadas como altas; de 0,40 até abaixo de 0,85, como parciais. Os 130 textos incluídos ficaram abaixo de 0,40 e não tiveram o corpo integral localizado nas coletâneas. Essa triagem não demonstra ineditismo nem ausência de duplicação parcial em outras edições.

O cruzamento com 32 títulos da bibliografia da Academia Brasileira de Letras (ABL) [8] encontrou 24 caminhos diretamente correspondentes, seis casos sem equivalência integral verificada, uma tradução excluída e a coletânea *Correspondência* (1932), não localizada no pacote. Essas contagens descrevem o inventário, não uma equivalência entre arquivos, livros e edições. Assim, os resultados referem-se à amostra de trabalho identificada pelo SHA-256 no manifesto, sem generalização à obra completa.

Os textos da fonte foram decodificados de Windows-1252 e armazenados em UTF-8. A preparação removeu apenas os delimitadores técnicos externos do arquivo mestre, aplicou normalização Unicode NFC, converteu finais de linha para LF, removeu espaços finais e reduziu sequências de mais de duas linhas vazias a duas. Cabeçalhos, notas editoriais, índices e conteúdo literário foram preservados e, portanto, também integram a distribuição aprendida.

A divisão por documento, com semente 20260925, buscou proporções de 80%/10%/10% dos caracteres. Foram alocados 181 documentos ao treino (11.113.713 caracteres), 30 à validação (1.340.673) e 31 ao teste (1.245.563), correspondendo a 81,12%, 9,79% e 9,09%. A diferença decorre dos comprimentos desiguais dos documentos. A verificação de duplicação integral

Tabela I
HIPERPARÂMETROS DO TREINAMENTO.

Item	Configuração
Inicialização	do zero
Camadas/cabeças/dimensão	6/6/384; reduzido 4/4/256
Contexto / lote	256 tokens / 64
Dropout / bias	0,1 / desativado
Otimizador	AdamW; $\beta_1 = 0,9$, $\beta_2 = 0,95$
Taxa inicial / mínima	$3 \cdot 10^{-4}$ / $3 \cdot 10^{-5}$
Aquecimento / decaimento	100 / 5.000 iterações
Weight decay / clipping	0,1 / 1,0
Precisão / compilação	float16 / desativada
Iterações / GPU	5.000 / Tesla T4

após normalização de espaços não encontrou grupos duplicados; isso não exclui passagens repetidas entre obras distintas.

O tokenizador foi ajustado somente no treino e possui 147 tokens, incluindo EOS e UNK. Houve zero caracteres mapeados para UNK no treino, dois na validação e um no teste. Cada documento é codificado caractere a caractere e terminado por EOS; os fluxos de cada partição são concatenados. Uma janela pode atravessar fronteiras entre documentos, e textos mais longos oferecem mais posições possíveis para amostragem.

B. Modelo e treinamento

Foi utilizado o nanoGPT, de Andrej Karpathy, sob licença MIT, no commit `3adf61e154c3` [3]. O baseline tem seis camadas, seis cabeças e dimensão de embedding 384; a condição reduzida tem quatro camadas, quatro cabeças e dimensão 256. Os demais hiperparâmetros são compartilhados, conforme a Tabela I. Ambas as configurações foram treinadas do zero por 5.000 iterações no Colab, com PyTorch 2.11.0+cu128 e Tesla T4.

O treinamento calcula gradientes por retropropagação e atualiza os pesos com AdamW. A semente efetiva do código `train.py` em uma GPU é 1337; já a preparação dos dados e a geração usam 20260925. A opção `always_save_checkpoint=True` grava o estado do treinamento (*checkpoint*) em cada avaliação. Os arquivos `ckpt.pt` preservados correspondem à iteração 5.000. As curvas registram avaliações a cada 500 iterações e mostram que esse checkpoint teve a menor perda de validação entre os pontos registrados.

Reduzir camadas, cabeças e dimensão simultaneamente constitui uma intervenção composta: os resultados comparam duas configurações, mas não isolam o efeito de cada componente. Há uma execução por condição. O orçamento iguala atualizações e tokens processados, não operações de ponto flutuante (FLOPs) nem tempo de execução.

C. Avaliação

A avaliação abre os documentos de teste somente após o treinamento de cada modelo. Usa 200 lotes de 32 janelas de 256 tokens, sorteadas do fluxo concatenado de teste com semente NumPy 20260926. Cada lote contém entradas e alvos deslocados em um token. Calcula-se a média das 200 perdas de lote e aplica-se a exponencial para obter a perplexidade. Trata-se de uma estimativa amostral, não de uma varredura

Tabela II
 AVALIAÇÃO AMOSTRAL. PERDA EM NATS POR TOKEN DO VOCABULÁRIO DE CARACTERES, INCLUINDO EOS E UNK; PPL CALCULADA SOBRE OS MESMOS TOKENS. PARÂMETROS INCLUEM POSIÇÕES.

Modelo	Parâmetros totais	Perda	PPL
Baseline (6/6/384)	10,78 M	1,2397	3,455
Reduzido (4/4/256)	3,25 M	1,3862	4,000

exaustiva nem de uma média por obra. Os inícios podem se repetir ou gerar janelas sobrepostas; as 6.400 sequências não são necessariamente independentes. A semente torna o sorteio reproduzível, mas não remove a incerteza amostral.

Ambas as condições foram avaliadas no Colab em Tesla T4, com PyTorch 2.11.0+cu128, a mesma semente de teste e o mesmo protocolo. O conjunto de teste contém uma ocorrência do caractere “½”, ausente do vocabulário ajustado no treino; ela foi mapeada para UNK e permaneceu como alvo na avaliação. O resultado é, portanto, uma comparação pareada no ambiente, embora baseada em uma amostra de janelas.

A geração usa o prefixo “Era ” (incluindo o espaço final), temperatura 0,8, top-k 40 e semente 20260925. São gerados 700 novos tokens, e o texto apresentado é truncado no primeiro EOS, quando presente. A temperatura reescala os escores antes da amostragem; top-k restringe a distribuição aos 40 candidatos de maior pontuação. O prefixo é um início narrativo, não uma pergunta. As amostras servem à inspeção qualitativa e não substituem avaliação humana sistemática de qualidade, autoria ou factualidade.

IV. RESULTADOS

Os dois checkpoints foram carregados com sucesso e registraram 5.000 iterações. A contagem total inclui embeddings posicionais e conta uma vez os pesos dos embeddings de token compartilhados com a projeção de saída: 10.776.576 no baseline e 3.251.200 na variante, redução de 69,8%. Sem posições, o nanoGPT informa 10.678.272 e 3.185.664 parâmetros. A perda amostral aumentou 0,1465 nats por token (11,8%) na configuração menor, e a perplexidade aumentou 0,545 (15,8%). As diferenças foram calculadas com a precisão integral dos resultados armazenados, antes do arredondamento da Tabela II.

Na amostra baseline, “Era uma coisa de Fernando recebida na sua imaginação” combina palavras e pontuação plausíveis, mas não estabelece relação semântica clara; a continuação mistura datas e nomes. Na amostra reduzida, “as duas amigas” e “a cadeira um ar de tendência” ilustram grafia inventada e continuidade instável. As duas gerações reproduzem pontuação e moldes de diálogo reconhecíveis, sem manter coerência narrativa. Esta inspeção é ilustrativa, não uma avaliação humana sistemática, e não sustenta uma conclusão global sobre qualidade literária.

A. Comparação experimental

Sob o protocolo compartilhado, o baseline apresenta menor perda e perplexidade amostrais; a condição reduzida utiliza menos parâmetros. A relação exponencial entre as métricas

permite expressar a diferença de perdas como uma razão de perplexidades:

$$\frac{PPL_{red}}{PPL_{base}} = \exp(L_{red} - L_{base}) \approx 1,158. \quad (3)$$

Portanto, a perplexidade amostral da variante é cerca de 15,8% maior. Sem erro-padrão nem repetições com sementes independentes, a incerteza não está quantificada. Os valores descrevem estas execuções, mas não estimam com precisão o desempenho esperado em novos treinamentos.

A redução do número de pesos implica menor armazenamento dos parâmetros sob a mesma precisão numérica. Não foram medidos latência, energia, taxa de processamento ou pico de memória em protocolo controlado. Por isso, a redução paramétrica de 69,8% não deve ser interpretada como ganho equivalente de velocidade ou economia computacional medida.

V. DISCUSSÃO

Os resultados indicam um compromisso entre número de parâmetros e qualidade preditiva: o modelo menor requer menos pesos, mas atribui menor probabilidade média geométrica aos tokens-alvo do teste. A menor perda do baseline não se traduz, nas amostras apresentadas, em continuidade narrativa consistente. Assim, a métrica de previsão local e a inspeção da geração descrevem aspectos distintos do comportamento dos modelos.

O corpus e a divisão são tão importantes quanto a arquitetura. Duplicatas ou sobreposição entre documentos podem reduzir artificialmente a dificuldade do teste; por isso, a auditoria de proveniência e a divisão por obra devem permanecer parte do protocolo. Estudos sobre deduplicação mostram que sobreposição de dados pode afetar tanto a avaliação quanto a memorização de modelos de linguagem [9].

As curvas de treino e validação das execuções atuais estão preservadas junto aos logs no Drive; a perda de validação diminuiu nos pontos registrados até a iteração 5.000. Esse resultado permite descrever a trajetória amostrada, embora não substitua repetições independentes para estimar variabilidade entre treinamentos. O checkpoint reportado é o da iteração 5.000, que também foi o de menor perda entre os pontos de avaliação registrados.

Há três limites principais à interpretação. Primeiro, a composição do pacote NLTK e a seleção heurística dos contos delimitam a representatividade da amostra. Segundo, a estimativa por janelas sobrepostas não quantifica a incerteza da perda; além disso, a ocorrência de “½” foi representada pelo token UNK. Terceiro, a intervenção arquitetural composta e a execução única por condição impedem separar seus componentes da variabilidade entre treinamentos. As duas condições atuais foram avaliadas no mesmo ambiente e protocolo; resultados de rodadas históricas em outros ambientes não são usados como comparação principal.

Há também uma limitação temporal: o resultado do baseline já era conhecido quando se decidiu executar a variante. O teste não foi usado para calcular gradientes, mas a escolha da segunda configuração não foi cega aos resultados anteriores. A

comparação deve, portanto, ser interpretada como exploratória. Em novos estudos, as configurações podem ser fixadas antes de consultar o teste, com decisões de arquitetura baseadas na validação.

VI. EXTENSÃO PARA PERGUNTAS SOBRE MACHADO DE ASSIS

Os modelos autorregressivos treinados apenas para continuação de texto não devem ser apresentados como sistemas de perguntas e respostas. Uma extensão possível seria combinar recuperação de passagens e geração. As obras seriam segmentadas e indexadas com título, capítulo e posição; diante de uma pergunta, o sistema recuperaria trechos relevantes e os forneceria ao gerador junto com instruções para responder com base nas evidências. A resposta deveria citar os trechos recuperados ou declarar insuficiência de evidência.

Por exemplo, diante de “Como Bentinho interpreta a ausência de Capitu?”, o sistema deveria localizar passagens pertinentes antes de formular a resposta. O contexto atual de 256 tokens de caracteres é restrito para acomodar pergunta, instrução e vários excertos. Seria necessário ampliar o contexto ou substituir/adaptar o gerador, além de avaliar sua capacidade de usar as evidências. Essa proposta se relaciona à geração aumentada por recuperação (RAG), mas recuperar trechos e inseri-los no prefixo não reproduz o treinamento conjunto do sistema original de Lewis et al. [10].

Um primeiro experimento dessa extensão poderia usar respostas extrativas, compostas pelas próprias passagens recuperadas, como referência antes de introduzir geração. A avaliação mediria precisão e cobertura dos trechos relevantes, apoio textual das respostas, atendimento à pergunta e abstenção apropriada, com perguntas anotadas e critérios definidos antes do teste. Esta é uma proposta, não uma funcionalidade implementada.

VII. CONCLUSÃO

Foram implementadas e comparadas duas configurações autorregressivas em nível de caractere. Sob o mesmo orçamento de atualizações, a variante reduzida tem 69,8% menos parâmetros únicos, incluindo posições, mas apresenta perda amostral 11,8% maior e perplexidade 15,8% maior. A comparação é descritiva e restrita às execuções e à amostra selecionada. As gerações exibem padrões locais de linguagem e falhas de coerência, sem demonstrar capacidade conversacional.

O experimento articula preparação rastreável de um corpus literário, treinamento do zero e avaliação dos limites preditivos e qualitativos de modelos pequenos. A preservação de curvas de validação e a avaliação com repetições fortaleceriam estudos futuros. Para perguntas sobre Machado de Assis, uma extensão possível é combinar recuperação de trechos e geração com evidências explícitas, avaliando separadamente os dois componentes.

REPRODUTIBILIDADE E ATRIBUIÇÃO

Os arquivos `model.py`, `train.py` e utilitários são do nanoGPT, de Andrej Karpathy, sob licença MIT; a configuração

de caracteres deriva de `shakespeare_char` [3]. Os notebooks do projeto acrescentam auditoria e preparação do corpus, tokenizador, partições, configurações, avaliação e exportação. Não se atribui autoria própria ao código-base.

Os notebooks executados e as instruções de reprodução estão no README do projeto. Os quatro notebooks abrem no Colab: 01 auditoria, 02 preparação, 03 baseline e 04 comparativo. Dados e resultados estão na pasta compartilhada do Google Drive; o PDF final também está disponível nela. Página de entrada do projeto: README e instruções de avaliação.

REFERÊNCIAS

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention Is All You Need,” 2017, versão arXiv; ano da submissão inicial; PDF v7 consultado em 25 set. 2026. [Online]. Available: <https://arxiv.org/abs/1706.03762>
- [2] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, “Language Models are Unsupervised Multitask Learners,” OpenAI, Tech. Rep., 2019. [Online]. Available: <https://cdn.openai.com/better-language-models/language-models.pdf>
- [3] A. Karpathy, “nanoGPT,” código-fonte, licença MIT, commit 3adf61e154c3. Acesso em: 26 set. 2026. [Online]. Available: <https://github.com/karpathy/nanoGPT/tree/3adf61e154c3fe3fca428ad6bc3818b27a3b8291>
- [4] —, “Let’s build GPT: from scratch, in code, spelled out.” YouTube, vídeo didático. Acesso em: 26 set. 2026. [Online]. Available: <https://www.youtube.com/watch?v=kCc8FmEb1nY>
- [5] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, “Language Models are Few-Shot Learners,” 2020, versão arXiv; ano da submissão inicial; PDF v4 consultado em 25 set. 2026. [Online]. Available: <https://arxiv.org/abs/2005.14165>
- [6] M. de Assis, “Machado de Assis – Obra Completa,” NLTK Data, pacote machado, metadados da distribuição digital; fonte declarada: MEC. Acesso em: 26 set. 2026. [Online]. Available: https://github.com/nltk/nltk_data/blob/gh-pages/packages/corpora/machado.xml
- [7] Brasil. Ministério da Educação and Universidade Federal de Santa Catarina. NUPILL, “Machado de Assis: vida e obra,” coleção digital. Acesso em: 25 set. 2026. [Online]. Available: <https://machado.mec.gov.br/>
- [8] Academia Brasileira de Letras, “Machado de Assis: bibliografia,” acesso em: 26 set. 2026. [Online]. Available: <https://www.academia.org.br/academicos/machado-de-assis/bibliografia>
- [9] K. Lee, D. Ippolito, A. Nystrom, C. Zhang, D. Eck, C. Callison-Burch, and N. Carlini, “Deduplicating Training Data Makes Language Models Better,” in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Dublin, Ireland: Association for Computational Linguistics, 2022, pp. 8424–8445. [Online]. Available: <https://aclanthology.org/2022.acl-long.577/>
- [10] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” 2020, versão arXiv; ano da submissão inicial; PDF v4 consultado em 25 set. 2026. [Online]. Available: <https://arxiv.org/abs/2005.11401>